

WEEK-6

Session-11,12

TASK-1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define HISTORY_SIZE 5

#define INPUT_SIZE 256

char history[HISTORY_SIZE][INPUT_SIZE];

int history_count = 0;

int next_position = 0;

void store_command(const char *command)

{

    if (command == NULL || strlen(command) == 0)

        return;

    snprintf(history[next_position], INPUT_SIZE, "%s", command);

    next_position = (next_position + 1) % HISTORY_SIZE;

    if (history_count < HISTORY_SIZE)

        history_count++;

}

void display_history(void)

{

    if (history_count == 0)

    {

        printf("History is empty.\n");

        return;

    }

}
```

```

int oldest = (next_position - history_count + HISTORY_SIZE)
    % HISTORY_SIZE;

printf("\n--- Command History ---\n");
for (int i = 0; i < history_count; i++)
{
    int index = (oldest + i) % HISTORY_SIZE;
    printf("%d %s\n", i + 1, history[index]);
}
printf("-----\n");
}

void get_history_entry(int number)
{
    if (number < 1 || number > history_count)
    {
        printf("Invalid history entry.\n");
        return;
    }

    int oldest = (next_position - history_count + HISTORY_SIZE)
        % HISTORY_SIZE;

    int index = (oldest + number - 1) % HISTORY_SIZE;
    printf("History %d: %s\n", number, history[index]);
}

void validate_history(void)
{
    int valid = 1;

    if (history_count < 0 || history_count > HISTORY_SIZE)
        valid = 0;

    if (next_position < 0 || next_position >= HISTORY_SIZE)
        valid = 0;

    if (valid)
        printf("History consistency: VALID\n");
}

```

```

else
    printf("History consistency: INVALID\n");
}
void clear_history(void)
{
    history_count = 0;
    next_position = 0;

    for (int i = 0; i < HISTORY_SIZE; i++)
        history[i][0] = '\0';

    printf("History cleared.\n");
}
int main(void)
{
    char input[INPUT_SIZE];
    printf("=== Command History Buffer ===\n");
    printf("History capacity: %d commands\n", HISTORY_SIZE);
    printf("\nCommands:\n");
    printf(" history    - Display history\n");
    printf(" get N        - Retrieve entry N\n");
    printf(" check        - Validate history\n");
    printf(" clear        - Clear history\n");
    printf(" exit         - Exit program\n");
    while (1)
    {
        printf("\nhistory-shell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)

```

```

        continue;
    if (strcmp(input, "history") == 0)
    {
        display_history();
    }
    else if (strcmp(input, "check") == 0)
    {
        validate_history();
    }
    else if (strcmp(input, "clear") == 0)
    {
        clear_history();
    }
    else if (strcmp(input, "exit") == 0)
    {
        printf("Exiting history shell.\n");
        break;
    }
    else if (strncmp(input, "get ", 4) == 0)
    {
        int number;

        if (sscanf(input + 4, "%d", &number) == 1)
            get_history_entry(number);
        else
            printf("Usage: get N\n");
    } else{
        store_command(input);
        printf("Command stored.\n");
    }
}

return 0;

```

}

```
root@LAPTOP-RJFDEBT:~# make command_history.c
root@LAPTOP-RJFDEBT:~# gcc -Wall -Wextra -g command_history.c -o command_history
== Command History Buffer ==
History capacity: 5 commands

Commands:
  history      - Display history
  get N        - Retrieve entry N
  check        - Validate history
  clear        - Clear history
  exit         - Exit program

history-shell> ls
Command stored.

history-shell> pwd
Command stored.

history-shell> date
Command stored.

history-shell> whoami
Command stored.

history-shell> uname
Command stored.

history-shell> echo hello
Command stored.

history-shell> history
-- Command History --
1  pwd
2  date
3  whoami
4  uname
5  echo hello

history-shell> get 1
History 1: pwd

history-shell> get 4
History 4: uname

history-shell> get 6
Invalid history entry.

history-shell> uname
Command stored.

history-shell> echo hello
Command stored.

history-shell> history
-- Command History --
1  pwd
2  date
3  whoami
4  uname
5  echo hello

history-shell> get 1
History 1: pwd

history-shell> get 4
History 4: uname

history-shell> get 6
Invalid history entry.

history-shell> uname
Command stored.

history-shell> echo hello
Command stored.

history-shell> history
-- Command History --
1  pwd
2  date
3  whoami
4  uname
5  echo hello

history-shell> get 1
History 1: pwd

history-shell> get 4
History 4: uname

history-shell> get 6
Invalid history entry.

history-shell> exit
Exiting history shell.
root@LAPTOP-RJFDEBT:~# ls -l command_history.c command_history
-rwxr-xr-x 1 root root 28528 Sep 18 04:55 command_history
-rw-r--r-- 1 root root 3589 Sep 18 04:55 command_history.c
root@LAPTOP-RJFDEBT:~# ./command_history
== Command History Buffer ==
History capacity: 5 commands

Commands:
  history      - Display history
  get N        - Retrieve entry N
  check        - Validate history
  clear        - Clear history
  exit         - Exit program

history-shell> |
```

TASK-2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#define MAX_COMMANDS 10
```

```
#define MAX_ARGS 20
```

```
#define INPUT_SIZE 256
```

```
typedef struct
```

```
{
```

```
    char command[INPUT_SIZE];
```

```
    char *args[MAX_ARGS];
```

```
    int arg_count;
```

```
} Command;
```

```
typedef struct
```

```
{
```

```
    Command commands[MAX_COMMANDS];
```

```
    int command_count;
```

```

} Pipeline;

void remove_newline(char *str)
{
    str[strcspn(str, "\n")] = '\0';
}

void parse_command(Command *cmd)
{
    char temp[INPUT_SIZE];
    strcpy(temp, cmd->command);
    cmd->arg_count = 0;
    char *token = strtok(temp, " ");
    while (token != NULL && cmd->arg_count < MAX_ARGS - 1)
    {
        cmd->args[cmd->arg_count] = strdup(token);
        cmd->arg_count++;
        token = strtok(NULL, " ");
    }
    cmd->args[cmd->arg_count] = NULL;
}

void add_command(Pipeline *pipeline, const char *command)
{
    if (pipeline->command_count >= MAX_COMMANDS)
    {
        printf("Pipeline capacity exceeded.\n");
        return;
    }
    strcpy(pipeline->commands[pipeline->command_count].command, command);
    parse_command(&pipeline->commands[pipeline->command_count]);
    pipeline->command_count++;
}

void display_pipeline(const Pipeline *pipeline)
{
    printf("\n===== PIPELINE LAYOUT =====\n");

```

```

if (pipeline->command_count == 0)
{
    printf("Pipeline is empty.\n");
    return;
}

for (int i = 0; i < pipeline->command_count; i++)
{
    printf("Stage %d: %s", i + 1,
        pipeline->commands[i].command);
    if (i < pipeline->command_count - 1)
        printf(" | ");
    printf("\n");
}

printf("=====\n");
}

int validate_pipeline(const Pipeline *pipeline)
{
    if (pipeline->command_count <= 0)
    {
        printf("Validation: INVALID - pipeline is empty.\n");
        return 0;
    }

    if (pipeline->command_count > MAX_COMMANDS)
    {
        printf("Validation: INVALID - too many commands.\n");
        return 0;
    }

    for (int i = 0; i < pipeline->command_count; i++)
    {
        if (strlen(pipeline->commands[i].command) == 0)
        {
            printf("Validation: INVALID - empty command at stage %d.\n",

```

```

        i + 1);
    return 0;
}
}

printf("Validation: VALID\n");
printf("Number of stages: %d\n", pipeline->command_count);
return 1;
}

void execute_pipeline(Pipeline *pipeline)
{
    int n = pipeline->command_count;
    if (n == 0)
    {
        printf("Nothing to execute.\n");
        return;
    }
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];
    for (int i = 0; i < n - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }
    for (int i = 0; i < n; i++)
    {
        pids[i] = fork();
        if (pids[i] == -1)
        {
            perror("fork");
            return;
        }
    }
}

```

```

    }
    if (pids[i] == 0)
    {
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        if (i < n - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }
        for (int j = 0; j < n - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }
        execvp(pipeline->commands[i].args[0],
            pipeline->commands[i].args);
        perror("execvp");
        exit(EXIT_FAILURE);
    }
}

for (int i = 0; i < n - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < n; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nPipeline execution completed.\n");
}

```

```

void free_pipeline(Pipeline *pipeline)
{
    for (int i = 0; i < pipeline->command_count; i++)
    {
        for (int j = 0;
             j < pipeline->commands[i].arg_count;
             j++)
        {
            free(pipeline->commands[i].args[j]);
        }
    }
}

int main(void)
{
    Pipeline pipeline;
    pipeline.command_count = 0;
    char input[INPUT_SIZE];
    printf("=====\n");
    printf("    OSSP PIPELINE SIMULATOR    \n");
    printf("=====\n");
    printf("\nEnter pipeline commands.\n");
    printf("Type 'run' to execute.\n");
    printf("Type 'show' to display pipeline.\n");
    printf("Type 'check' to validate pipeline.\n");
    printf("Type 'clear' to clear pipeline.\n");
    printf("Type 'exit' to quit.\n\n");
    while (1) {
        printf("pipeline> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        remove_newline(input);
        if (strlen(input) == 0)
            continue;
    }
}

```

```

    if (strcmp(input, "exit") == 0)
    {
        break;
    }
    else if (strcmp(input, "show") == 0)
    {
        display_pipeline(&pipeline);
    }
    else if (strcmp(input, "check") == 0)
    {
        validate_pipeline(&pipeline);
    }
    else if (strcmp(input, "run") == 0)
    {
        if (validate_pipeline(&pipeline))
            execute_pipeline(&pipeline);
    }
    else if (strcmp(input, "clear") == 0)
    {
        free_pipeline(&pipeline);
        pipeline.command_count = 0;
        printf("Pipeline cleared.\n");
    } else {
        add_command(&pipeline, input);
        printf("Command stored as stage %d.\n",
            pipeline.command_count);
    }
    free_pipeline(&pipeline);
    printf("\nExiting pipeline simulator.\n");
    return 0;
}

```

```

root@laptop-ajf08e8t:~# make pipeline.o
root@laptop-ajf08e8t:~# gcc -Wall -Wextra -g pipeline.c -o pipeline
root@laptop-ajf08e8t:~# ./pipeline
=====
OSIP PIPELINE SIMULATOR
=====
Enter pipeline commands.
Type 'run' to execute.
Type 'show' to display pipeline.
Type 'check' to validate pipeline.
Type 'clear' to clear pipeline.
Type 'exit' to quit.

pipeline ls
Command stored as stage 1.
pipeline wc -l
Command stored as stage 2.
pipeline show
===== PIPELINE LAYOUT =====
Stage 1: ls |
Stage 2: wc -l
=====
pipeline check
Validation: VALID
Number of stages: 2
pipeline run
Validation: VALID
Number of stages: 2
wc: invalid option -- 'l'
Try 'wc --help' for more information.
Pipeline execution completed.
pipeline clear
Pipeline cleared.
pipeline ls
Command stored as stage 1.
pipeline wc -l
Command stored as stage 2.
pipeline show
Command stored as stage 3.
pipeline show
===== PIPELINE LAYOUT =====
Stage 1: ls |
Stage 2: wc -l |
Stage 3: wc -l
=====
pipeline check
Validation: VALID
Number of stages: 3
pipeline run
Validation: VALID
Number of stages: 3
wc: No such file or directory
Pipeline execution completed.
pipeline exit

Exiting pipeline simulator.
root@laptop-ajf08e8t:~# ls -l pipeline.o pipeline
-rw-r--r-- 1 root root 2220 Sep 18 05:15 pipeline.o
-rw-r--r-- 1 root root 6194 Sep 18 05:05 pipeline
root@laptop-ajf08e8t:~# valgrind --leak-check=full ./pipeline
==2280== Memcheck, a memory error detector
==2280== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2280== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2280== Command: ./pipeline
==2280==

=====
OSIP PIPELINE SIMULATOR
=====
Enter pipeline commands.
Type 'run' to execute.
Type 'show' to display pipeline.
Type 'check' to validate pipeline.
Type 'clear' to clear pipeline.
Type 'exit' to quit.

pipeline exit

Exiting pipeline simulator.
==2280==
==2280== HEAP SUMMARY:
==2280==    in use at exit: 0 bytes in 0 blocks
==2280==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==2280==
==2280== All heap blocks were freed -- no leaks are possible
==2280==
==2280== For lists of detected and suppressed errors, rerun with: -s
==2280== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 free p)
root@laptop-ajf08e8t:~#

```

TASK-3

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

Code:

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main(void)
{
    int pipe1[2];

    int pipe2[2];

    pid_t p1, p2, p3;

    if (pipe(pipe1) == -1)
    {
        perror("pipe1");
        exit(EXIT_FAILURE);
    }

    if (pipe(pipe2) == -1)
    {
        perror("pipe2");
        exit(EXIT_FAILURE);
    }
}

```

```

}

printf("Pipes created successfully.\n");

printf("Pipe 1: read=%d, write=%d\n", pipe1[0], pipe1[1]);
printf("Pipe 2: read=%d, write=%d\n", pipe2[0], pipe2[1]);

p1 = fork();
if (p1 == -1)
{
    perror("fork");
    exit(EXIT_FAILURE);
}
if (p1 == 0)
{
    dup2(pipe1[1], STDOUT_FILENO);
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    execlp("ls", "ls", "-l", (char *)NULL);
    perror("execlp ls");
    exit(EXIT_FAILURE);
}

p2 = fork();

if (p2 == -1)
{
    perror("fork");
    exit(EXIT_FAILURE);
}
if (p2 == 0)
{
    dup2(pipe1[0], STDIN_FILENO);

```

```

    dup2(pipe2[1], STDOUT_FILENO);
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    execlp("grep", "grep", "^d", (char *)NULL);
    perror("execlp grep");
    exit(EXIT_FAILURE);
}

p3 = fork();
if (p3 == -1)
{
    perror("fork");
    exit(EXIT_FAILURE);
}
if (p3 == 0)
{
    dup2(pipe2[0], STDIN_FILENO);
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    execlp("wc", "wc", "-l", (char *)NULL);
    perror("execlp wc");
    exit(EXIT_FAILURE);
}
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);
waitpid(p1, NULL, 0);

```

}

[illegible]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <string.h>
```

```
#define MAX_COMMANDS 10
```

```
#define MAX_ARGS 20
```

```
typedef struct
```

```
{
```

```
    char *args[MAX_ARGS];
```

```
} Command;
```

```
/* Free allocated command arguments */
```

```
void free_commands(Command commands[], int count)
```

```
{
```

```
    for (int i = 0; i < count; i++)
```

```
    {
```

```
        for (int j = 0; commands[i].args[j] != NULL; j++)
```

```
        {
```

```
            free(commands[i].args[j]);
```

```
        }
```

```
    }
```

```
}
```

```
int main(void)
```

```
{
```

```
    /*
```

```
    Long pipeline:
```

```
    printf -> grep -> sort -> uniq -> wc
```

```
*/
```

```
Command commands[5] =
```

```
{
```

```
    {"printf", "apple\nbanana\napple\norange\nbanana\napple\n", NULL},
```

```
    {"grep", "apple", NULL},
```

```
    {"sort", NULL},
```

```
    {"uniq", NULL},
```

```
    {"wc", "-l", NULL}
```

```
};
```

```
int command_count = 5;
```

```
/* Allocate copies of command arguments */
```

```
for (int i = 0; i < command_count; i++)
```

```
{
```

```
    for (int j = 0; commands[i].args[j] != NULL; j++)
```

```
    {
```

```
        char *copy = strdup(commands[i].args[j]);
```

```
        if (copy == NULL)
```

```
        {
```

```
            perror("strdup");
```

```
            free_commands(commands, command_count);
```

```
            exit(EXIT_FAILURE);
```

```
        }
```

```
        commands[i].args[j] = copy;
```

```
    }
```

```
}
```

```

/*
    For N processes, N-1 pipes are required.

    Here:

    5 processes -> 4 pipes
*/

int pipes[MAX_COMMANDS - 1][2];
pid_t pids[MAX_COMMANDS];

printf("=====\n");
printf("    MULTIPLE PIPE SIMULATOR    \n");
printf("=====\n");

printf("\nNumber of processes: %d\n", command_count);
printf("Number of pipes: %d\n\n", command_count - 1);

/* Display pipeline */
for (int i = 0; i < command_count; i++)
{
    printf("Stage %d: ", i + 1);

    for (int j = 0; commands[i].args[j] != NULL; j++)
    {
        printf("%s ", commands[i].args[j]);
    }

    if (i < command_count - 1)
        printf(" | ");

    printf("\n");
}

```

```

printf("\nCreating pipes...\n");

/* Create multiple pipes */
for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");
        free_commands(commands, command_count);
        exit(EXIT_FAILURE);
    }

    printf("Pipe %d created: read=%d write=%d\n",
        i + 1, pipes[i][0], pipes[i][1]);
}

printf("\nLaunching processes...\n");

/* Create processes */
for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] == -1)
    {
        perror("fork");
        free_commands(commands, command_count);
        exit(EXIT_FAILURE);
    }

    if (pids[i] == 0)

```

```

{
    /*
        If this is not the first process,
        connect input to previous pipe.
    */
    if (i > 0)
    {
        if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
        {
            perror("dup2 input");
            exit(EXIT_FAILURE);
        }
    }

    /*
        If this is not the last process,
        connect output to next pipe.
    */
    if (i < command_count - 1)
    {
        if (dup2(pipes[i][1], STDOUT_FILENO) == -1)
        {
            perror("dup2 output");
            exit(EXIT_FAILURE);
        }
    }

    /*
        Close every pipe descriptor in the child.
    */
    for (int j = 0; j < command_count - 1; j++)

```

```

    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    /* Execute command */
    execvp(commands[i].args[0], commands[i].args);

    perror("execvp");
    exit(EXIT_FAILURE);
}

printf("Process %d launched with PID %d\n",
       i + 1, pids[i]);
}

/*
    Parent no longer needs any pipe descriptors.
    Close all of them.
*/
printf("\nClosing pipe descriptors in parent...\n");

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/*
    Wait for all processes.
*/

```

```

printf("Waiting for all processes...\n");

for (int i = 0; i < command_count; i++)
{
    int status;

    waitpid(pids[i], &status, 0);

    if (WIFEXITED(status))
    {
        printf("Process %d completed with status %d\n",
            i + 1, WEXITSTATUS(status));
    }
}

free_commands(commands, command_count);

printf("\n=====\\n");
printf("Long pipeline completed successfully.\\n");
printf("All pipes closed.\\n");
printf("All processes synchronized.\\n");
printf("Resources cleaned up.\\n");
printf("=====\\n");

return 0;

```

```

root@LAPTOP-RJFDBEET:~# name multi_pipe.c
root@LAPTOP-RJFDBEET:~# gcc -Wall -x86_64 -o multi_pipe.c -o multi_pipe
root@LAPTOP-RJFDBEET:~# ./multi_pipe
=====
MULTIPLE PIPE SIMULATOR
=====
Number of processes: 5
Number of pipes: 4

Stage 1: printf apple
banana
apple
orange
banana
apple

Stage 2: grep apple |
Stage 3: sort |
Stage 4: uniq |
Stage 5: wc -l

Creating pipes...
Pipe 1 created: read=3 write=0
Pipe 2 created: read=3 write=0
Pipe 3 created: read=3 write=0
Pipe 4 created: read=9 write=10

Launching processes...
Process 1 launched with PID 7088
Process 2 launched with PID 7089
Process 3 launched with PID 7010
Process 4 launched with PID 7011
Process 5 launched with PID 7012

Closing pipe descriptors in parent...
Waiting for all processes...
Process 1 completed with status 0
Process 2 completed with status 0
Process 3 completed with status 0
Process 4 completed with status 0
Process 5 completed with status 0

=====
Long pipeline completed successfully.
All pipes closed.
All processes synchronized.
Resources cleaned up.
=====
root@LAPTOP-RJFDBEET:~# printf "apple|banana|apple|orange|banana|apple|n" | grep apple | sort | uniq | wc -l
1
root@LAPTOP-RJFDBEET:~# strace -f -e trace=pipe,pipe2,dup2,close,clone,fork,vfork,execve,wait* ./multi_pipe
execve("./multi_pipe", ["./multi_pipe"], 0x7ffdbdb118 /* 27 vars */) = 0
close(3) = 0
close(4) = 0
close(5) = 0
close(6) = 0
close(7) = 0
close(8) = 0
close(9) = 0
close(10) = 0
close(11) = 0
close(12) = 0
close(13) = 0
close(14) = 0
close(15) = 0
close(16) = 0
close(17) = 0
close(18) = 0
close(19) = 0
close(20) = 0
close(21) = 0
close(22) = 0
close(23) = 0
close(24) = 0
close(25) = 0
close(26) = 0
close(27) = 0
close(28) = 0
close(29) = 0
close(30) = 0
close(31) = 0
close(32) = 0
close(33) = 0
close(34) = 0
close(35) = 0
close(36) = 0
close(37) = 0
close(38) = 0
close(39) = 0
close(40) = 0
close(41) = 0
close(42) = 0
close(43) = 0
close(44) = 0
close(45) = 0
close(46) = 0
close(47) = 0
close(48) = 0
close(49) = 0
close(50) = 0
close(51) = 0
close(52) = 0
close(53) = 0
close(54) = 0
close(55) = 0
close(56) = 0
close(57) = 0
close(58) = 0
close(59) = 0
close(60) = 0
close(61) = 0
close(62) = 0
close(63) = 0
close(64) = 0
close(65) = 0
close(66) = 0
close(67) = 0
close(68) = 0
close(69) = 0
close(70) = 0
close(71) = 0
close(72) = 0
close(73) = 0
close(74) = 0
close(75) = 0
close(76) = 0
close(77) = 0
close(78) = 0
close(79) = 0
close(80) = 0
close(81) = 0
close(82) = 0
close(83) = 0
close(84) = 0
close(85) = 0
close(86) = 0
close(87) = 0
close(88) = 0
close(89) = 0
close(90) = 0
close(91) = 0
close(92) = 0
close(93) = 0
close(94) = 0
close(95) = 0
close(96) = 0
close(97) = 0
close(98) = 0
close(99) = 0
close(100) = 0
close(101) = 0
close(102) = 0
close(103) = 0
close(104) = 0
close(105) = 0
close(106) = 0
close(107) = 0
close(108) = 0
close(109) = 0
close(110) = 0
close(111) = 0
close(112) = 0
close(113) = 0
close(114) = 0
close(115) = 0
close(116) = 0
close(117) = 0
close(118) = 0
close(119) = 0
close(120) = 0
close(121) = 0
close(122) = 0
close(123) = 0
close(124) = 0
close(125) = 0
close(126) = 0
close(127) = 0
close(128) = 0
close(129) = 0
close(130) = 0
close(131) = 0
close(132) = 0
close(133) = 0
close(134) = 0
close(135) = 0
close(136) = 0
close(137) = 0
close(138) = 0
close(139) = 0
close(140) = 0
close(141) = 0
close(142) = 0
close(143) = 0
close(144) = 0
close(145) = 0
close(146) = 0
close(147) = 0
close(148) = 0
close(149) = 0
close(150) = 0
close(151) = 0
close(152) = 0
close(153) = 0
close(154) = 0
close(155) = 0
close(156) = 0
close(157) = 0
close(158) = 0
close(159) = 0
close(160) = 0
close(161) = 0
close(162) = 0
close(163) = 0
close(164) = 0
close(165) = 0
close(166) = 0
close(167) = 0
close(168) = 0
close(169) = 0
close(170) = 0
close(171) = 0
close(172) = 0
close(173) = 0
close(174) = 0
close(175) = 0
close(176) = 0
close(177) = 0
close(178) = 0
close(179) = 0
close(180) = 0
close(181) = 0
close(182) = 0
close(183) = 0
close(184) = 0
close(185) = 0
close(186) = 0
close(187) = 0
close(188) = 0
close(189) = 0
close(190) = 0
close(191) = 0
close(192) = 0
close(193) = 0
close(194) = 0
close(195) = 0
close(196) = 0
close(197) = 0
close(198) = 0
close(199) = 0
close(200) = 0
close(201) = 0
close(202) = 0
close(203) = 0
close(204) = 0
close(205) = 0
close(206) = 0
close(207) = 0
close(208) = 0
close(209) = 0
close(210) = 0
close(211) = 0
close(212) = 0
close(213) = 0
close(214) = 0
close(215) = 0
close(216) = 0
close(217) = 0
close(218) = 0
close(219) = 0
close(220) = 0
close(221) = 0
close(222) = 0
close(223) = 0
close(224) = 0
close(225) = 0
close(226) = 0
close(227) = 0
close(228) = 0
close(229) = 0
close(230) = 0
close(231) = 0
close(232) = 0
close(233) = 0
close(234) = 0
close(235) = 0
close(236) = 0
close(237) = 0
close(238) = 0
close(239) = 0
close(240) = 0
close(241) = 0
close(242) = 0
close(243) = 0
close(244) = 0
close(245) = 0
close(246) = 0
close(247) = 0
close(248) = 0
close(249) = 0
close(250) = 0
close(251) = 0
close(252) = 0
close(253) = 0
close(254) = 0
close(255) = 0
close(256) = 0
close(257) = 0
close(258) = 0
close(259) = 0
close(260) = 0
close(261) = 0
close(262) = 0
close(263) = 0
close(264) = 0
close(265) = 0
close(266) = 0
close(267) = 0
close(268) = 0
close(269) = 0
close(270) = 0
close(271) = 0
close(272) = 0
close(273) = 0
close(274) = 0
close(275) = 0
close(276) = 0
close(277) = 0
close(278) = 0
close(279) = 0
close(280) = 0
close(281) = 0
close(282) = 0
close(283) = 0
close(284) = 0
close(285) = 0
close(286) = 0
close(287) = 0
close(288) = 0
close(289) = 0
close(290) = 0
close(291) = 0
close(292) = 0
close(293) = 0
close(294) = 0
close(295) = 0
close(296) = 0
close(297) = 0
close(298) = 0
close(299) = 0
close(300) = 0
close(301) = 0
close(302) = 0
close(303) = 0
close(304) = 0
close(305) = 0
close(306) = 0
close(307) = 0
close(308) = 0
close(309) = 0
close(310) = 0
close(311) = 0
close(312) = 0
close(313) = 0
close(314) = 0
close(315) = 0
close(316) = 0
close(317) = 0
close(318) = 0
close(319) = 0
close(320) = 0
close(321) = 0
close(322) = 0
close(323) = 0
close(324) = 0
close(325) = 0
close(326) = 0
close(327) = 0
close(328) = 0
close(329) = 0
close(330) = 0
close(331) = 0
close(332) = 0
close(333) = 0
close(334) = 0
close(335) = 0
close(336) = 0
close(337) = 0
close(338) = 0
close(339) = 0
close(340) = 0
close(341) = 0
close(342) = 0
close(343) = 0
close(344) = 0
close(345) = 0
close(346) = 0
close(347) = 0
close(348) = 0
close(349) = 0
close(350) = 0
close(351) = 0
close(352) = 0
close(353) = 0
close(354) = 0
close(355) = 0
close(356) = 0
close(357) = 0
close(358) = 0
close(359) = 0
close(360) = 0
close(361) = 0
close(362) = 0
close(363) = 0
close(364) = 0
close(365) = 0
close(366) = 0
close(367) = 0
close(368) = 0
close(369) = 0
close(370) = 0
close(371) = 0
close(372) = 0
close(373)
```

